

Hack Hydra — Competition Brief

Source: <https://hackhydra.hydradb.com> · Repo: <https://github.com/hydra-db/hydradb> · Discord: <https://discord.gg/D8cGSa9H9> · Register: <https://luma.com/h038glzk> · Submit: <https://forms.gle/WEwqEmmN7Bkp4HyJ6>

At a glance

Event	Hack Hydra — nine-day online hackathon, run by HydraDB
Dates	Aug 12 – Aug 20, 2026
Deadline	Aug 20, 2026, 11:59 PM PT (hard; form closes)
Winners	Aug 24, 2026
Prizes	\$10,000 total — \$5,000 / \$3,000 / \$1,500 + \$500 special
Team size	1–4 (one team per person; solo allowed)
Cost / format	Free · 100% online
Core constraint	HydraDB must do real work , not sit in the README

Today is Aug 13 — roughly 7½ days of build time remain. The occasion for this brief is that the clock is already running.

Where this repo stands

CHECK	STATUS
First commit on/after Aug 12	YES cc125a1, 2026-08-13
Public GitHub repo	YES
Open-source license	YES MIT, on main
Registered on Luma	YES done
HydraDB running locally	YES Docker, smoke test 15/15
Track chosen	NO pending — see recommendation

What HydraDB actually is

This matters more than usual, because one of the five judging criteria is literally "use of HydraDB and graph-native approaches," and a \$500 award exists purely for using it well. A project that treats it as a generic Neo4j substitute scores badly on the one axis the organizers care most about.

- **Distributed graph database written in Rust**, storing data durably in **S3-compatible object storage** (built on SlateDB). Storage and compute are fully separated.
- **Two tiers**. `graph-node` executes queries and mutations with optional local SSD/NVMe cache. `graph-indexer` asynchronously builds immutable traversal indexes and publishes them via atomic object-store pointers. Both keep only disposable state locally, so they scale and recover independently.
- **OpenCypher** query language — the same as Neo4j.
- **Three ways in**: Bolt on 7687 (any Neo4j driver — Python, JS/Node, Java, Go, Ruby, .NET), HTTP JSON/NDJSON on 8443, admin + Prometheus metrics on 9090.
- **Native server-side path procedures**, which avoid client-side query fan-out:
 - `algo.SPpaths` — single source -> single target
 - `algo.SSpaths` — single source -> many targets
 - `algo.MSpaths` — many sources -> many targets, pairwise
- **GraphBLAS sparse-linear-algebra kernel** for traversal.
- **Snapshot-consistent reads**. Every query runs against one pinned SlateDB snapshot with WAL overlays. Consistency modes: `causal` (default) and `strong`.
- **License: AGPL-3.0**. Rust 1.91+, needs `libcypher-parser` and SuiteSparse GraphBLAS.

Fastest path to running: the Docker image `ghcr.io/hydra-db/hydradb:latest` with `CLOUD_PROVIDER=local`. Building from source needs the full C/C++ toolchain plus `just native-check` / `just smoke`.

Licensing note (not legal advice): HydraDB is AGPL-3.0. We plan to run it as a separate containerized service and talk to it over Bolt/HTTP as a client, which keeps our own code out of the derivative-work question. Our repo should carry a permissive license (MIT or Apache-2.0) to satisfy the "open-source license" requirement.

The three `algo.*paths` procedures and the object-storage snapshot model are the differentiators to build on. Anything we can only do because traversal happens server-side, or because reads pin a consistent snapshot, is exactly the "hard to pull off with traditional vector or relational approaches" that the Best Use award names.

Running locally, and what v0.1.0 actually supports

HydraDB is now running in this repo — `./hydradb/up.sh`, verified by `./hydradb/smoke-test.sh` (15/15). Full setup notes and the complete capability matrix are in [hydradb/README.md](#). Three findings from probing the running build matter for planning:

1. **The Cypher surface is a narrow subset.** `CREATE` only accepts one-hop edge patterns, so standalone node creation fails. Aggregates including `count()` are unsupported. Bare `MATCH (n)` is rejected — every match needs an id, label, or property anchor. `UNWIND` is refused by the query transport, and because list parameters only bind through `UNWIND`, **every list argument must be inlined as a literal** in the query string.
1. **Reverse variable-length traversal is rejected** — variable-length `MATCH` requires a fixed source id. This means `MATCH (x)-[:DEPENDS_ON*1..n]->(victim)`, the reverse-dependency closure that is the blast-radius question, cannot be expressed directly. The fix is a data-model decision: write every dependency edge **in both directions** at ingest (`DEPENDS_ON` forward, `USED_BY` reverse) and traverse forward from the compromised package. Verified working against the live instance.
1. **All three native path procedures work**, and return full path objects with nodes and relationships. Signatures, which are not documented anywhere and were recovered by probing:

```
CALL algo.SPpaths({sourceNode: 100, targetNode: 103, relTypes: ["DEPENDS_ON"]}) YIELD path
CALL algo.SSpaths({sourceNode: 100, relTypes: ["DEPENDS_ON"]}) YIELD path
CALL algo.MSpaths({sourceLabel: "Pkg", sourceProperty: "name", sourceValues: ["app-a"],
  targetLabel: "Pkg", targetProperty: "name", targetValues: ["lib-b"],
  relTypes: ["DEPENDS_ON"]}) YIELD path
```

`SPpaths` and `SSpaths` address nodes by vertex id; `MSpaths` resolves them by property lookup and requires all six keys.

This confirms rather than weakens the Track 02A recommendation. The blast-radius query is expressible today on a bidirectional edge model, and the engine limits push work toward exactly the graph-native design the judges are scoring. It also means anyone who assumes full Cypher will lose a day to it — knowing the subset now is a real head start, and the recovered `algo.*` signatures are worth contributing back to the repo or sharing in Discord.

The three tracks

Every track is framed the same way: a problem that is **a graph problem, not a semantic-similarity problem**. Each submission enters under exactly one track. The top submission from each track advances to a final round of three.

Track 01 — Enterprise context and ontology

Build an ontology out of real enterprise applications.

You get roughly **half a million documents** from nine sources, arriving with all the noise a real company has: misfiled documents, near-duplicates, and statements that flatly contradict each other.

SOURCE	APPROX. DOCS	CONTENT
Slack	275,000	Internal team discussion
Gmail	120,000	Management + leadership mail
Linear	35,000	Engineering / product tickets
Google Drive	25,000	Collaborative documents
HubSpot	15,000	Sales CRM records
Fireflies	10,000	Meeting transcripts
GitHub	8,000	Repository activity
Jira	6,000	Support tickets
Confluence	5,000	Wikis and documentation

What to build: turn that corpus into a clean, queryable ontology in HydraDB, then answer questions spanning simple lookups, multi-hop reasoning, conflict resolution, and correctly recognizing when the answer simply is not in the corpus.

The hard part, stated by the organizers: extraction is easy now that LLMs are cheap — don't over-invest there. The difficulty is **entity resolution and ontology alignment**: deciding that "Sam," "@soham," and "S. Ratnaparkhi" are one

person, and figuring out which of two contradictory statements to trust.

Datasets

- **EnterpriseRAG-Bench** (Onyx) — the 500k-document corpus above. 500 questions across 10 types, of which 175 are "Basic"; a separate set covers metadata-dependent questions. Scored on **retrieval accuracy** — whether the system locates the correct source documents. Deliberately seeded with internal terminology, stale information, misfiling, and near-duplicates. Distributed via GitHub releases and HuggingFace, including per-source slices. Has a leaderboard requiring reproducible results.
- **Salesforce HERB** — 28.9 MB, simulating product planning -> development -> support. A `metadata/` folder (customer profiles, org structure, employees) and a `products/` folder (one JSON per product with team assignments, customer associations, and artifacts). Artifacts include Slack messages, meeting transcripts, meeting chats, documents, URLs, pull requests, and both **answerable** and **unanswerable** questions. Multi-hop with guaranteed ground truth. Note for RAG evaluation: infer team and customer information from the artifacts, not from direct field mappings.

Read: the highest-ceiling track and the one closest to HydraDB's actual commercial pitch. Also the most expensive — 500k documents is real ingestion time and real LLM spend, and the per-source slices exist for a reason.

Track 02 — Repos, dependencies and code as graphs

Search the graph, and catch chained vulnerabilities before they land.

Supply chain attacks through npm and PyPI are surging, and developer tools today fail to give real-time, deep context on malicious dependencies. When a package is compromised, the questions that matter are:

- Which internal services are transitively exposed?
- Which version of the dependency introduced the vulnerability?
- Which applications resolved the compromised version **while it was live**?
- Which other packages share maintainers or infrastructure with it?
- Are there likely typosquat packages nearby?
- What is the complete blast radius?

The central insight, in the organizers' words: this is fundamentally a graph traversal and dependency problem, not a semantic similarity problem.

Pick **A** or **B**.

Option A — Supply chain blast radius

The motivating incident: in the **TanStack compromise this May**, 84 malicious package artifacts were published across 42 packages within **six minutes** of the CI pipeline being breached. The worm went on to hit Mistral AI, UiPath, and over 160 other npm and PyPI packages — self-propagating, and persisting in `.claude/` and `.vscode/` directories in a way that survived `npm uninstall`.

The defender's problem is **speed**: a package is compromised at 09:00 — which of your services are exposed by 09:06? That is a transitive reverse-dependency closure over an ecosystem graph with **tens of millions of versioned nodes**, and the organizers note flatly that a vector index cannot answer it at all.

Build the npm or PyPI dependency graph in HydraDB and answer that. Then go further: which packages share a maintainer with the compromised one; which lockfiles resolved to the bad version during the window it was live; and which names sit close enough to a popular package to be a typosquat.

Option B — Code graphs for IDE assistants

Every IDE assistant embeds repositories and retrieves chunks by similarity, and **similarity is a weak proxy for relevance**. Code needs call chains, type definitions, startup configs, and tests. Build a code graph in HydraDB and serve better context from it — described as "the missing piece for editor wrappers today."

Read: Option A is the best structural fit for HydraDB in the whole competition. The data is free and public, the core query is a `lgo.MSpaths` over a reverse-dependency closure, "resolved the bad version while it was live" is a temporal-snapshot question, and the demo is visceral. Option B is a more crowded space and needs tree-sitter/LSP plumbing before any graph work starts.

Track 03 — Memory and context retrieval

Make your own mem0, and ace the benchmarks.

Build an agent memory layer for cross-session continuity. It has to process chat histories spanning **30–40 sessions and 115,000 tokens per question**.

The system must synthesize facts across sessions, keep chronological order, and track information that was **later overwritten**. Long-context models drop **30–60% in accuracy** here, and they mostly fail at **abstention**: knowing when the answer simply is not in the history, and saying so instead of inventing one.

Datasets

- **LongMemEval** (ICLR 2025) — five core abilities: information extraction, multi-session reasoning, knowledge updates, temporal reasoning, and abstention. Question types: `single-session-user`, `single-session-assistant`, `single-session-preference`, `temporal-reasoning`, `knowledge-update`, `multi-session`, with `_abs` suffixes marking abstention variants. Three variants, 500 instances each: `_S` (~40 sessions, ~115k tokens — this is the one the track description quotes), `_M` (~500 sessions), `_Oracle` (evidence sessions only, an upper bound). Evaluated at turn level (memory recall) and session level (evidence-session identification), with a GPT-4o judge for QA correctness; outputs are JSONL with `question_id` and `hypothesis`. Retrieval baselines provided: `flat-bm25`, `flat-contriever`, `flat-stella`, `flat-gte`.
- **LongMemEval V2** — the updated release.
- **BEAM** — 100 multi-domain conversations (coding, math, health, finance) with ~2,000 validated probing questions. Length distribution: `20 × 128K` tokens, `35 × 500K`, `35 × 1M`, `10 × 10M`. Ten memory abilities: abstention, contradiction resolution, event ordering, information extraction, instruction following, knowledge update, multi-session reasoning, preference following, summarization, temporal reasoning. Pipeline: `answer_generation.sh -> run_evaluation` (LLM-as-judge) -> `report_results.py`.

Read: the easiest track to start and therefore likely the most crowded, and the one where the organizers' "not just benchmark scores" caveat bites hardest — you would need to beat the benchmarks *and* ship a product. Judge-model API spend is non-trivial.

How judging works

Round 1 — within your track. Projects are evaluated against the other submissions in their own track. The top submission from each track advances.

Round 2 — final. Judges compare the three finalists holistically and rank them for Grand Champion, Runner-Up, and Third Place.

Structural consequence: **only three projects total reach the final round, one per track.** Track choice is partly a competition-density bet, not just a fit question. Winning a less-crowded track is worth more than placing second in a popular one — second place inside a track wins nothing.

The five criteria

1. Technical execution
2. Use of HydraDB and graph-native approaches
3. Product completeness and usability
4. Quality of results
5. Originality

A strong submission has: a functional product or demo · real ingestion **and** retrieval workflows · a clear use case · a thoughtful technical implementation.

"We care about working, thoughtful products, not just benchmark scores."

Best Use of HydraDB — \$500

Judged separately, can go to any eligible submission including a finalist. Criteria:

- A particularly strong graph data model
- A novel retrieval or reasoning approach
- An interesting use of relationships, traversal or context
- A use case that is hard to pull off with traditional vector or relational approaches

This is the most winnable award and it stacks with everything else — every team stays eligible regardless of track.

Multiple tracks

Allowed, but each entry must be a **meaningfully distinct project**; the same project with minor modifications does not count. A team can be a finalist in more than one track but takes home only one of the top three awards. You may switch tracks mid-build as long as the final submission clearly fits the track it is submitted under.

What we have to submit

All three by **Aug 20, 11:59 PM PT**. The form closes on time; late entries are not accepted unless an extension is announced.

1. Submission form

<https://forms.gle/WEwqEmmN7Bkp4HyJ6> — asks for:

- Project name
- Short project description
- Problem being addressed
- What you built

- Deployed project link, if available
- **How the project uses the HydraDB Open Source Repo**
- Tech stack used
- Team members and individual contributions
- GitHub repository link
- 3-minute demo video link

2. Demo video — 3 minutes or less

Must cover: the problem · what you actually built · **a demo of the project working** · how you used the HydraDB repo and why it matters.

YouTube is fine, unlisted is fine, as long as judges can watch without requesting access. **Anything past the 3-minute mark may not be reviewed** — so the working demo has to land early, not after ninety seconds of setup.

3. Public GitHub repository

Must contain:

- | |
|--|
| ☐ Complete source code |
| ☐ No participant-authored commits before Aug 12, 2026 |
| ☐ A clear README |
| ☐ Setup and run instructions |
| ☐ An explanation of how HydraDB is used |
| ☐ Required environment / dependency information |
| ☐ Attribution for third-party libraries, APIs, datasets, open-source code |
| ☐ An open-source license <- we do not have this yet |

Pre-existing commits from upstream repos, libraries, templates, and dependencies do **not** count against us. Judges may review contribution history to verify the work was created during the event.

Rules and disqualification

Rules

- Work starts on or after Aug 12, 2026 — fresh repo, because judges read commit history.
- Existing open-source libraries, frameworks, APIs, public datasets, and **AI coding assistants are all explicitly allowed**. The submitted project must still be substantially built during the event.
- HydraDB must do real work. Be ready to say where it is used and what the project would lose without it.
- Teams of 1–4, every member listed on the form, one team per person.
- Suggested datasets are optional unless a track requires one; own or other public datasets are fine if disclosed in the README.
- You retain ownership of what you build, subject to your license and third-party terms.
- HydraDB employees may build alongside but are not eligible for prizes.
- HydraDB may withhold a prize if submissions do not meet a minimum quality bar.

Disqualification triggers

- Work started before Aug 12, 2026
- Missing or private GitHub repository
- No open-source license in the repository
- Missing demo video
- HydraDB not used meaningfully
- Submitted after the deadline
- Breaking the rules or code of conduct

Organizers explicitly warn: **open your repo, video, and demo links yourself before submitting — broken links are the most common way people lose.**

Recommendation

Take Track 02, Option A — supply chain blast radius. Reasoning:

1. **Best graph-native fit.** The headline query is a transitive reverse-dependency closure — literally what a `lgo.MSpaths` and the GraphBLAS traversal kernel exist for. Criterion #2 is the one we can most credibly max out, and it doubles as the Best Use of HydraDB pitch.
2. **Cheapest data.** npm registry, PyPI, deps.dev, OSV, and GitHub Advisory are all free and public. No 500k-document ingestion bill, no LLM-judge spend. In a 7½-day window, data acquisition cost is a schedule risk, and this track has almost none.
3. **"Vector databases cannot do this" is handed to us.** The organizers say it in the track text. That is criterion #2 and the Best Use rubric quoting themselves.
4. **The demo is visceral.** "This package was compromised at 09:00 — here is your exposure at 09:06" is a three-minute video that sells itself. Compare with explaining entity-resolution F1 in the same time.
5. **Temporal versioning is a natural depth axis.** "Which applications resolved the compromised version while it was live" maps onto HydraDB's pinned-snapshot read model — an obvious place to go beyond a baseline and into originality.

Fallback: Track 01, if we want the largest surface area and are willing to absorb the ingestion cost — start with per-source slices rather than the full 500k, and spend our effort on entity resolution, which is where the organizers said the difficulty lives.

Avoid: Track 03 unless we have a genuinely novel memory architecture. It is the easiest track to enter, which makes it the most crowded, and beating benchmarks alone is explicitly not what wins.

Rough shape of the remaining time

DAYS	FOCUS
~~Aug 13~~	YES Registered; MIT license on <code>main</code> ; HydraDB running via Docker; Bolt + HTTP round-trip validated; <code>algo.*paths</code> signatures recovered
Aug 14–15	Lock the graph data model — bidirectional edges are now a known requirement; build the ingestion pipeline; load a real slice of the ecosystem
Aug 15–17	Core queries — reverse-dependency closure, maintainer overlap, version-window resolution, typosquat proximity
Aug 17–19	Product surface: the thing judges actually click. Real ingestion and retrieval workflows, per the rubric
Aug 19	README, setup instructions, HydraDB usage writeup, attribution, license check
Aug 20	Record and cut the 3-minute video; verify every link from a logged-out browser; submit early

Both of the day-one blockers are now cleared — the MIT license is on `main`, and HydraDB runs locally with a passing smoke test. **The one remaining decision is the track**, and it gates the data model, so it should be settled before any ingestion code is written.